TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ
TRUYỀN THÔNG VIỆT – HÀN
KHOA KHOA HỌC MÁY TÍNH

---- ----

VKU

ĐỒ ÁN MÔN HỌC:
LINUX VÀ PHẦN MỀM NGUỒN MỞ

ĐỀ TÀI: Configure and Deploy MLOps (Machine Learning Ops) Services on Arch Linux & Ubuntu

Sinh viên thực hiện : Nguyễn Công Đức (Trưởng nhóm - Arch Linux Node)

Nguyễn Tùng Dương (Thành viên - Ubuntu Infra & Security)

Thành viên 3 (Thành viên - Observability & Benchmark)

Giảng viên hướng dẫn : TS. Hoàng Hữu Đức

Đà Nẵng, tháng 10 năm 2026

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến TS. Hoàng Hữu Đức – người thầy đã tận tình hướng dẫn, truyền đạt kiến thức và định hướng cho nhóm trong suốt thời gian thực hiện đồ án môn học Linux và Phần mềm nguồn mở.

Đề tài "Configure and Deploy MLOps Services on Arch Linux & Ubuntu" đã giúp chúng em có cơ hội nghiên cứu sâu sắc về hạt nhân Linux, các kỹ thuật phân quyền cgroups v2, User Namespaces và sự khác biệt giữa hai trường phái hệ điều hành: Rolling Release (Arch Linux) và LTS (Ubuntu Server).

Chúng em cũng xin gửi lời cảm ơn đến quý Thầy Cô Khoa Khoa Học Máy Tính, Trường Đại học Công nghệ Thông tin và Truyền thông Việt – Hàn đã trang bị nền tảng kiến thức vững chắc cho chúng em.

Chúng em xin chân thành cảm ơn!

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

.....
.....

Đà Nẵng, ngày .... tháng 10 năm 2026

Chữ ký giảng viên hướng dẫn

TS. Hoàng Hữu Đức

MỤC LỤC

LỜI CẢM ƠN 2

NHẬN XÉT CỦA GIẢNG VIÊN 3

MỤC LỤC 4

DANH MỤC HÌNH ẢNH 6

DANH MỤC BẢNG BIỂU 7

MỞ ĐẦU 8

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI 10

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ KIẾN TRÚC HỆ THỐNG 17

CHƯƠNG 3: THIẾT KẾ VÀ QUY TRÌNH TRIỂN KHAI IAC 30

CHƯƠNG 4: TRIỂN KHAI VÀ THỰC NGHIỆM ĐỐI KHÁNG 42

CHƯƠNG 5: ĐÁNH GIÁ ĐỊNH LƯỢNG BENCHMARK VÀ BẢO MẬT 58

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 68

TÀI LIỆU THAM KHẢO 73

MỞ ĐẦU

Trong làn sóng chuyển đổi số và sự bùng nổ của Trí tuệ nhân tạo (AI), các hệ thống MLOps đóng vai trò huyết mạch kết nối giữa mô hình học máy và môi trường vận hành sản xuất. Tuy nhiên, việc vận hành MLOps trên hạ tầng Linux đặt ra những bài toán hóc búa về kiến trúc hệ điều hành và an toàn thông tin.

Một câu hỏi lớn trong giới kỹ sư hệ thống là: Nên lựa chọn Arch Linux (Rolling Release - cập nhật liên tục, tự do tối giản) hay Ubuntu Server (LTS - ổn định dài hạn, doanh nghiệp)? Hai trường phái này mang lại sự đánh đổi (trade-off) như thế nào về độ an toàn của container, tính năng phân quyền kernel và độ trễ phục vụ suy luận?

Đặc biệt, việc phần lớn các hệ thống container hiện nay vẫn chạy dưới quyền root (UID 0) đặt ra mối nguy cơ thoát rào container (Container Escape) đe dọa trực tiếp tới máy chủ host. Đề tài tập trung triển khai và đánh giá so sánh thực nghiệm một cụm MLOps chuẩn mực trên cả hai nền tảng, áp dụng công nghệ Rootless Docker unprivileged trên Arch Linux nhằm loại bỏ hoàn toàn nguy cơ chiếm quyền root.

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1. ĐẶT VẤN ĐỀ

Các tác vụ Machine Learning ngày nay đòi hỏi một chuỗi liên kết các công cụ mã nguồn mở: MLflow làm máy chủ theo dõi thí nghiệm, PostgreSQL lưu trữ quan hệ metadata, MinIO lưu trữ đối tượng nhị phân mô hình S3, FastAPI phục vụ suy luận vi dịch vụ (Inference Microservice), và Prometheus/Grafana giám sát tài nguyên.

Trên Ubuntu Server, mô hình cài đặt truyền thống cấp quyền root cho Docker daemon. Ngược lại, trên Arch Linux với triết lý "The Arch Way", chúng ta có thể làm chủ hạt nhân (Kernel), tinh chỉnh sysctl somaxconn, và triển khai Rootless Docker tận dụng tối đa User Namespaces và cgroups v2.

1.2. MỤC TIÊU ĐỀ TÀI

Đề tài hướng tới việc xây dựng hai node độc lập, tự động hóa 100% bằng Bash Script (IaC), triển khai cụm MLOps phân tách mạng Zero-Trust và xây dựng công cụ kiểm định, đo tải đối sánh định lượng.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ KIẾN TRÚC HỆ THỐNG

2.1. TỔNG QUAN HỆ ĐIỀU HÀNH LINUX & NHÂN KERNEL

Kiến trúc hạt nhân Linux quản lý 5 thành phần cốt lõi: Process Management, Memory Management, File System (overlay2), Network Stack (netfilter), và Security Subsystem.

Trong đề tài, hạt nhân Arch Linux (Kernel 7.x) cung cấp tính năng phân quyền unprivileged cgroups v2 hoàn hảo, cho phép phiên người dùng mlops tự quản lý hạn mức tài nguyên mà không cần tới quyền sudo.

2.2. CÔNG NGHỆ CONTAINER VÀ ROOTLESS DOCKER

Khác với Docker truyền thống chạy bằng daemon root, Rootless Docker khởi chạy dockerd hoàn toàn bên trong User Namespace. Bằng việc cấu hình dải UID trong /etc/subuid (mlops:100000:65536), tiến trình root (UID 0) bên trong container thực chất chỉ là một user vô danh (UID 100000) trên máy thật. Đây là bức tường thép ngăn chặn triệt để kỹ thuật tấn công container escape.

2.3. TƯỜNG LỬA STATEFUL: NFTABLES VS UFW

Trên Arch Linux, đề tài triển khai trực tiếp nftables bản địa với cấu trúc bảng inet filter, tích hợp cơ chế dynamic set đo lường tần suất kết nối (rate-limiting) chặn đứng brute-force SSH. Trên Ubuntu Server, tường lửa UFW được thiết lập với chính sách mặc định chặn toàn bộ kết nối đi vào (deny incoming).

CHƯƠNG 3: THIẾT KẾ VÀ QUY TRÌNH TRIỂN KHAI IAC

3.1. QUY CHUẨN TỰ ĐỘNG HÓA VÀ NGUYÊN TẮC IDEMPOTENT

Toàn bộ các kịch bản triển khai đều tuân thủ nguyên tắc lập trình shell phòng thủ: sử dụng set -euo pipefail, xây dựng hàm bẫy lỗi (trap) hiển thị chính xác dòng lệnh thất bại và tích hợp kịch bản rollback đưa hệ thống về nguyên trạng.

3.2. THIẾT KẾ MẠNG BẢO MẬT ZERO-TRUST

Hệ thống Docker Compose được thiết kế phân tách hai mạng riêng biệt:

- mlops-backend (internal: true): Mạng hoàn toàn cách ly, không cấp quyền ra internet và không ánh xạ port ra host. Cổng PostgreSQL 5432 và MinIO API 9000 nằm trọn trong ranh giới này.

- mlops-frontend: Mạng cầu nối công khai cho phép người dùng truy cập các dịch vụ: FastAPI (port 8080), MLflow UI (port 5000), Grafana (port 3000).

CHƯƠNG 4: TRIỂN KHAI VÀ THỰC NGHIỆM ĐỐI KHÁNG

4.1. QUÁ TRÌNH TRIỂN KHAI TRÊN ARCH LINUX NODE

1. Kịch bản 01_system_hardening.sh: Tự động khởi tạo người dùng mlops, áp dụng 99-mlops.conf sysctl (somaxconn=65535, tcp_tw_reuse=1), nạp cấu hình nftables.conf và kích hoạt SSH port 2222.

2. Kịch bản 02_setup_rootless_docker.sh: Thiết lập User Namespaces, cấp quyền systemd lingering qua loginctl enable-linger mlops và khởi chạy dockerd-rootless daemon.

3. Kịch bản 03_deploy_stack.sh: Cài đặt dịch vụ systemd cấp user (mlops-stack.service) và kích hoạt cụm Docker Compose 5 containers.

4. Ứng dụng AI Vision OCR & Công cụ Terminal CLI: Đóng gói FastAPI microservice phục vụ nhận diện chữ từ ảnh bằng engine Tesseract-OCR, tích hợp công cụ dòng lệnh ai_cli.sh cho phép tương tác trực tiếp trên Terminal.

4.2. QUÁ TRÌNH TRIỂN KHAI TRÊN UBUNTU SERVER NODE

Trên node Ubuntu Server, thành viên 2 đã triển khai thành công script hardening xử lý triệt để cơ chế ssh.socket, cấu hình tường lửa UFW chỉ mở đúng 5 port quy định và xây dựng cơ chế tự động backup/rollback an toàn.

CHƯƠNG 5: ĐÁNH GIÁ ĐỊNH LƯỢNG BENCHMARK VÀ BẢO MẬT

5.1. KẾT QUẢ KIỂM ĐỊNH BẢO MẬT HỆ THỐNG

Kịch bản tự động 04_verify_system.sh đã xác nhận 5/5 bài kiểm tra đạt chuẩn an toàn tuyệt đối:

- Tường lửa nftables: Đang hoạt động, chặn đứng rà quét cổng.

- Cơ chế cô lập Rootless: Container UID 0 ánh xạ chuẩn xác vào Host UID 1003 của user mlops.

- Kiểm tra cổng Zero-Trust: Quét thử cổng 5432 (Postgres) và 9000 từ host -> Kết nối bị từ chối (Connection Refused), chứng minh cơ sở dữ liệu được bảo vệ an toàn.

5.2. ĐÁNH GIÁ ĐỐI SÁNH HIỆU NĂNG (BENCHMARKING)

Thông qua kịch bản benchmark.sh gửi đồng thời hàng trăm request vào FastAPI Inference:

- Mức tiêu thụ RAM tĩnh (Idle Footprint): Arch Linux chỉ chiếm khoảng 450 MB RAM sau khi khởi động, thấp hơn đáng kể so với Ubuntu Server (~750 MB RAM) nhờ loại bỏ các dịch vụ chạy nền không cần thiết.

- Thông lượng phục vụ (Throughput): Cả hai hệ thống đều đạt trên 300 Requests/Second với độ trễ xử lý trung bình dưới 150 ms đối với tác vụ nhận diện chữ từ hình ảnh.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. KẾT LUẬN

Đề tài đã hoàn thành xuất sắc 100% các mục tiêu nghiên cứu và triển khai đề ra. Dự án chứng minh tính khả thi vượt trội của việc áp dụng Rootless Docker trên Arch Linux cho các khối lượng công việc MLOps hiện đại, đạt được sự cân bằng hoàn hảo giữa bảo mật hạt nhân và hiệu năng xử lý.

6.2. HƯỚNG PHÁT TRIỂN

1. Tích hợp mã hóa TLS cho toàn bộ các giao tiếp mạng giữa Docker daemon và API.

2. Mở rộng cụm phân tán Swarm/Kubernetes unprivileged đa nút tính toán.

3. Xây dựng hệ thống tự động cảnh báo gửi trực tiếp qua Telegram Bot khi phát hiện tấn công mạng.

TÀI LIỆU THAM KHẢO

[1] Docker Documentation - Run the Docker daemon as a non-root user (Rootless mode). https://docs.docker.com/engine/security/rootless/

[2] ArchWiki - Docker & nftables security guidelines.
https://wiki.archlinux.org/title/Docker

[3] Center for Internet Security (CIS) - CIS Docker Benchmark v1.6.0, 2024.

[4] MLflow Project Documentation - Production Deployment & Tracking Server.
https://mlflow.org/docs/latest/

[5] Prometheus & Grafana Labs - Cloud-Native Observability and Node Monitoring Architecture.

[6] Ubuntu Server Official Guide - OpenSSH Hardening and UFW Ruleset Management.